

Reassessment Problems – Loops: Part B

Learning Outcome Addressed

- Recognize, interpret, and write programs using while loops and for loops.
- Recognize, interpret, and write programs that iterate through lists and strings with for loops.
- Evaluate provided test sets and write new test sets to verify that code works as expected.

This assessment allows you to apply what you have learned in this module about while and for loops, using lists and strings with for loops, and assessing and writing test sets. Follow the instructions below to complete problems one to eight. If you get stuck on a problem without making any progress, please reach out to the learning facilitator for assistance.

1. [4 pts] `n_fours`

Write a function `n_fours(n)` that takes a non-negative integer `n` and returns an integer consisting of exactly `n` digits, where every digit is 4.

So `n_fours(0) == 0`, `n_fours(1) == 4`, `n_fours(2) == 44`, `n_fours(3) == 444`, and so on.

You may not use strings in this problem.

2. [4 pts] `shortest_string_length`

Write the function `shortest_string_length(L)` that takes a list of strings (e.g. `["hi", "hello", "howdy"]`) and returns the length of the shortest string in the list.

3. [8 pts] `num_even_digits`

Write the function `num_even_digits(n)` that takes in an integer `n` and returns the number of even digits in the number.

4. [12 pts] average_of_composites

Write the function **average_of_composites(L)** that takes in a list of integers and returns the average of all the composite numbers in the list. Numbers with more than 2 factors are called composite numbers (non-prime numbers).

For convenience, the `is_prime` function has been implemented for you. If you'd like an extra challenge, try writing it from scratch!

5. [12 pts] is_vowel

Write the function **is_vowel(s)** that takes in a string (e.g. "au 123") and returns True if there is at least one character in the string, and all the characters in the string are vowels. The function returns False otherwise.

Hint: If `c` denotes a character, then we can check if `c` is a vowel as follows: `c in "aeiouAEIOU"`

6. [12 pts] is_consonant

Write the function **is_consonant(s)** that takes in a string and returns True if every character in the string is a consonant (upper or lowercase) and there is at least one character in the string. The function returns False otherwise.

A vowel is one of the letters 'a', 'e', 'i', 'o', or 'u', while any other letter in the English alphabet is considered a consonant.

7. [20 pts] vowel_consonant_score

You are given a string **s** consisting of English letters, spaces, and digits.

Let **vowel_count** be the number of vowels in `s` and **cons_count** be the number of consonants in `s`.

A vowel is one of the letters 'a', 'e', 'i', 'o', or 'u' (lower or upper case), while any other letter in the English alphabet is considered a consonant.

The **score** of the string `s` is defined as follows:

- If `cons_count > 0`, the `score = vowels_count / cons_count`
round the answer to the nearest integer.
- Otherwise, the `score = 0`.

Return an integer denoting the score of the string.

Your function should call the `is_vowel` and `is_consonant` functions that you have already written.

For Example, if the Input `s = "cooear"`, then the output is 2.

The string `s = "cooear"` contains 4 vowels ('o', 'o', 'e', 'a') and 2 consonants ('c', 'r').

The score is $(\text{vowel_count} / \text{cons_count}) = (4 / 2) = 2$.

8. [8 pts] longest_coin_run

Write the function **longest_coin_run(s)** that takes in a string consisting of only 'H' (Head) and 'T' (Tail). The returned value is the length of the longest consecutive run of 'H' or 'T' in the string.

For example, if the string is:

`s = "HHTTTHTTTTTHH"`

then the output should be 5 since there is a run of 5 consecutive 'T' characters, and that is the longest run in the string.